

Цикл for в Python

Неумоин Павел Дмитриевич

6 апреля 2026 г.

Блиц-опрос

Вопрос 1. Что выведет программа?

```
x = 15
if x > 10:
    print("A")
elif x > 5:
    print("B")
else:
    print("C")
```

Вопрос 2. Что выведет программа?

```
a = 7
b = 3
if a % b == 1:
    print("Да")
else:
    print("Нет")
```

Блиц-опрос

Вопрос 3. Найдите ошибку:

```
n = int(input())  
if n % 2 = 0:  
    print("Чётное")
```

Вопрос 4. Что выведет программа?

```
x = 0  
if x:  
    print("Истина")  
else:  
    print("Ложь")
```

Блиц-опрос

Вопрос 5. Допишите условие:
вывести <<Подросток>>, если
возраст от 13 до 17 включительно.

```
age = int(input())  
if ???:  
    print("Подросток")
```

Вопрос 6. Что выведет программа?

```
a, b = 5, 10  
if a > 3 and b < 8:  
    print("Оба")  
elif a > 3:  
    print("Первое")  
else:  
    print("Ничего")
```

Блиц-опрос --- Ответы

1. **A** --- первое условие истинно, остальные не проверяются
2. **<<Да>>** --- $7 \div 3 = 2$ ост. 1, значит $7\%3 = 1$
3. Ошибка: = вместо ==
(присваивание вместо сравнения)
4. **<<Ложь>>** --- 0 интерпретируется как False
5. $13 \leq \text{age} \leq 17$ (или $\text{age} \geq 13$ and $\text{age} \leq 17$)
6. **<<Первое>>** --- условие $b < 8$ ложно, но $a > 3$ истинно

Историческая справка

Теория

Откуда взялось слово `<<for>>`?

Слово `for` в программировании впервые появилось в языке **FORTRAN** (1957 г.) --- одном из первых языков программирования в истории. Там цикл записывался как `DO 10 I = 1, 100`.

В современном Python цикл `for` кардинально отличается от C/C++/Java: он **перебирает элементы последовательности**, а не считает целые числа. Это делает его одним из самых мощных инструментов языка.

Факт: В Python **нет** классического цикла `for (i=0; i<n; i++)`. Вместо этого используется связка `for + range()`.

Синтаксис цикла for

Общий вид

for переменная **in** последовательность:телоцикла

Ключевые слова:

- for --- начало цикла
- in --- <<в>> (перебираем элементы)
- : --- двоеточие в конце строки!
- Отступ 4 пробела = тело цикла

■ Что можно перебирать?

- range() --- диапазон чисел
- list --- список
- str --- строку (посимвольно)
- tuple --- кортеж

Функция range() --- один аргумент

range(stop) --- генерирует числа от **0** до **stop** — 1

```
1 for i in range(5):  
2     print(i)
```

Вывод: 0 \n 1 \n 2 \n 3 \n 4

■ Запомни!

- Начинает с **0**, не с 1!
- Заканчивает **до** stop (не включая!)
- range(5) → 5 чисел: 0, 1, 2, 3, 4
- range(0) → пустой диапазон

Функция range() --- два аргумента

range(start, stop) --- числа от **start** до **stop** — 1

Пример 1:

```
1 for i in range(3, 8):  
2     print(i, end=" ")
```

Вывод: 3 4 5 6 7

Пример 2:

```
1 for i in range(1, 4):  
2     print(i * 10)
```

Вывод: 10, 20, 30

■ Математически

$\text{range}(a, b)$ = полуинтервал $[a; b)$

Левая граница **включена**,
правая **НЕ включена**!

Количество итераций: $b - a$

Частая ошибка:

```
# Хотимот 1 до 10 включительно:  
for i in range(1, 10): # 1..9!  
for i in range(1, 11): # 1..10
```

Функция range() --- три аргумента

range(start, stop, step) --- с шагом **step**

Пример 1 --- шаг 2 (чётные):

```
1 for i in range(0, 10, 2):  
2     print(i, end=" ")
```

Вывод: 0 2 4 6 8

Пример 2 --- шаг 3:

```
1 for i in range(1, 15, 3):  
2     print(i, end=" ")
```

Вывод: 1 4 7 10 13

Пример 3 --- обратный отсчёт:

```
1 for i in range(10, 0, -1):  
2     print(i, end=" ")
```

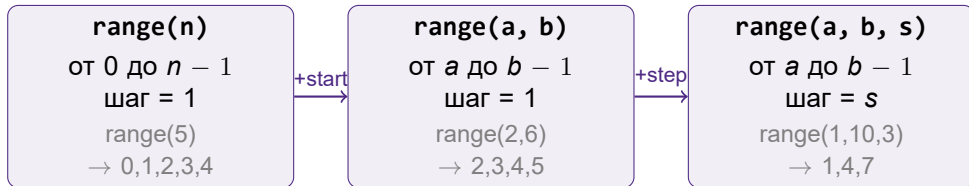
Вывод: 10 9 8 7 6 5 4 3 2 1

Пример 4 --- обратный с шагом -2:

```
1 for i in range(20, 0, -2):  
2     print(i, end=" ")
```

Вывод: 20 18 16 14 12 10 8 6 4 2

Итого: три формы range()



■ Золотое правило

`range()` **НИКОГДА** не включает правую границу!

Чтобы получить числа от 1 до N включительно: `range(1, N+1)`

Задача 1 --- Сумма чисел (условие)

Задача

Выведите сумму всех целых чисел от 1 до N (число N вводится с клавиатуры).

Пример: Ввод: 5 → Вывод: 15 (потому что $1 + 2 + 3 + 4 + 5 = 15$)

Задача 1 --- Сумма чисел (решение)

Решение

```
1 n = int(input())
2 suma = 0 # накопительсуммы
3 for i in range(1, n + 1): # от 1 до n включительно
4     suma = suma + i # или suma += i
5 print(suma)
```

Трассировка при $n = 5$:

Итерация	i	suma
начало	---	0
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

■ Паттерн <<Накопитель>>

1. Создать переменную = 0
2. В цикле прибавлять
3. После цикла --- результат

Работает и для **произведения**:

```
prod = 1
```

```
prod = prod * i
```

Задача 2 --- Таблица умножения (условие)

Задача

Выведите таблицу умножения на число k (вводится с клавиатуры), от $k \times 1$ до $k \times 10$.

Пример: Ввод: $7 \rightarrow 7 * 1 = 7, 7 * 2 = 14, \dots, 7 * 10 = 70$

Задача 2 --- Таблица умножения (решение)

Решение

```
1 k = int(input())
2 for i in range(1, 11):      # от 1 до 10 включительно
3     result = k * i
4     print(f"{k} * {i} = {result}")
```

7 * 1 = 7

7 * 2 = 14

7 * 3 = 21

...

7 * 10 = 70

■ f-строки

f"..." --- форматированная строка. Внутри {выражение} подставляется значение переменной. Удобнее, чем `print(k, "*", i, "=", result)`.

Цикл for со списком

Что такое список?

Список (list) --- упорядоченная коллекция элементов. Создается в квадратных скобках: [элемент1, элемент2, ...]

Перебор чисел:

```
1 nums = [10, 20, 30, 40]
2 for n in nums:
3     print(n)
```

Вывод: 10, 20, 30, 40

Перебор строк:

```
1 fruits = ["яблоко", "банан",
2           "вишня"]
3 for f in fruits:
4     print(f)
```

Вывод: яблоко, банан, вишня

Список vs Range --- в чём разница?

■ range() --- диапазон

- Только **целые числа**
- Только **арифм. прогрессия**
- Не хранит все числа в памяти
- Удобен для счётчиков

```
for i in range(1, 6):  
    print(i)           # 1 2 3 4 5
```

■ list --- список

- **Любые** типы данных
- **Произвольные** значения
- Хранит все элементы
- Удобен для данных

```
for x in [3, 1, 4, 1, 5]:  
    print(x)           # 3 1 4 1 5
```

Теория

Когда что использовать?

Нужно повторить N раз $\rightarrow$ range(N).
Нужно перебрать конкретные значения $\rightarrow$ список.

Полезные операции со списком в цикле

Сумма элементов:

```
1 nums = [4, 8, 15, 16, 23, 42]
2 s = 0
3 for x in nums:
4     s += x
5 print("Сумма:", s) # 108
```

Максимум:

```
1 nums = [4, 8, 15, 16, 23, 42]
2 mx = nums[0]
3 for x in nums:
4     if x > mx:
5         mx = x
6 print("Макс:", mx) # 42
```

Подсчёт по условию:

```
1 nums = [4, 8, 15, 16, 23, 42]
2 count = 0
3 for x in nums:
4     if x > 10:
5         count += 1
6 print("Больше 10:", count) # 4
```

Три паттерна

1. Накопитель суммы (`s += x`)
2. Поиск max/min (`if x > mx`)
3. Счётчик (`count += 1`)

Задача 3 --- Средняя оценка (условие)

Дан список оценок ученика: [5, 4, 3, 5, 4, 5, 3, 4].

Ожидаемый вывод: Средняя оценка: 4.1

Задача 3 --- Средняя оценка (решение)

Решение

```
1 grades = [5, 4, 3, 5, 4, 5, 3, 4]
2 summa = 0
3 for g in grades:
4     summa += g
5 average = summa / len(grades)
6 print(f"Средняя оценка: {round(average, 1)}")
```

Вывод: Средняя оценка: 4.1

■ Новые функции

- `len()` --- длина списка
- `round(x, 1)` --- округление до 1 знака

Формула:

$$\text{среднее} = \frac{\text{сумма элементов}}{\text{количество элементов}}$$

Задача 4 --- Фильтрация списка (условие)

Дан список чисел: [12, 5, -3, 8, -7, 0, 15, -1].

Ожидаемый вывод: 12, 5, 8, 15

Задача 4 --- Фильтрация списка (решение)

Решение

```
1 numbers = [12, 5, -3, 8, -7, 0, 15, -1]
2 for x in numbers:
3     if x > 0:                # фильтр: только положительные
4         print(x)
```

Вывод: 12, 5, 8, 15

■ Паттерн <<Фильтр>>

```
for элемент in список:
    if условие:
        # обрабатываем
```

Тело `if` выполняется НЕ на каждой итерации, а только когда условие истинно.

Цикл for со строкой --- основы

Строка = последовательность символов

В Python строка --- это **последовательность**, как и список. Цикл for перебирает её **посимвольно**: на каждой итерации переменная получает **один символ**.

Пример 1 --- посимвольный вывод:

```
1 word = "Python"
2 for ch in word:
3     print(ch)
```

Вывод: P, y, t, h, o, n

■ Что происходит?

Итерация	ch
1	"P"
2	"y"
3	"t"
4	"h"
5	"o"
6	"n"

ch --- просто имя переменной.

Цикл for со строкой --- примеры

Пример 2 --- с end:

```
1 for ch in "Привет":  
2     print(ch, end="-")
```

Вывод: П-р-и-в-е-т-

Пример 3 --- пробелы тоже символы!

```
1 for ch in "A B":  
2     print(f"[{ch}]", end=" ")
```

Вывод: [A] [] [B]

■ Важно!

Пробел " " --- это тоже символ.
Цикл его **не пропускает!**

Пример 4 --- подсчёт гласных:

```
1 text = "Информатика"  
2 vowels = "аеёиоуыэяАЕЁИОУЫЭЯ"  
3 count = 0  
4 for ch in text:  
5     if ch in vowels:  
6         count += 1  
7 print("Гласных:", count)
```

Вывод: Гласных: 5

Пример 5 --- длина строки:

```
1 s = "Hello"  
2 n = 0  
3 for ch in s:  
4     n += 1  
5 print(n)    # 5 то( же, что len(s))
```


Строка как последовательность

Что можно делать со строкой?

Индексация (нумерация с 0):

```
s = "Python"
# P y t h o n
# 0 1 2 3 4 5
print(s[0])      # P
print(s[3])      # h
print(s[-1])     # n с( конца)
```

Длина строки:

```
s = "Привет"
print(len(s))    # 6
```

Проверка вхождения (in):

```
s = "Информатика"
print("форм" in s)    # True
print("xyz" in s)     # False
```

■ Строка vs Список

- Строка --- **неизменяемая**
- Нельзя: `s[0] = "p"` → Ошибка!
- Можно собрать **новую** строку

Полезные операции со строкой в цикле

Подсчёт символов:

```
1 text = "Привет, мир!"
2 count = 0
3 for ch in text:
4     if ch == "и":
5         count += 1
6 print(f"Буква и': {count} раз")
```

Вывод: Буква 'и': 2 раз

Подсчёт цифр:

```
1 s = "abc123def45"
2 digits = 0
3 for ch in s:
4     if ch.isdigit():
5         digits += 1
6 print("Цифр:", digits) # 5
```

Сборка новой строки:

```
1 word = "Python"
2 result = ""
3 for ch in word:
4     result = ch + result
5 print(result) # nohtyP
```

Вывод: nohtyP (реверс!)

Замена символов:

```
1 phrase = "hello world"
2 new = ""
3 for ch in phrase:
4     if ch == " ":
5         new += "_"
6     else:
7         new += ch
8 print(new) # hello_world
```

Методы строк --- справочник

Полезные методы str

Метод	Описание	Пример ch = "A"
ch.isalpha()	Это буква?	True
ch.isdigit()	Это цифра?	False
ch.isspace()	Это пробел?	False
ch.isupper()	Заглавная?	True
ch.islower()	Строчная?	False
ch.upper()	В верхний регистр	"A"
ch.lower()	В нижний регистр	"a"

Пример --- перевод в верхний регистр:

```
word = "Abc"
for ch in word:
    if ch.isalpha():
        print(ch.upper(), end=" ")    # ABC
    else:
        print(ch, end=" ")
```

Методы строк --- практика

■ Выделяем гласные

```
vowels = "аеёиоуыэюя"
word = "информатика"
result = ""
for ch in word:
    if ch in vowels:
        result += ch.upper()
    else:
        result += ch
print(result)
```

Вывод: ИнфОрмАтИкА

■ Оператор in

`ch in "abc"` --- проверяет, содержит ли строка "abc" символ `ch`. Очень удобно!

■ Удаление цифр

```
s = "a1b2c3"
clean = ""
for ch in s:
    if not ch.isdigit():
        clean += ch
print(clean) # abc
```

Задача --- Анализ строки (условие)

Задача

Пользователь вводит строку. Программа должна подсчитать и вывести:

1. Количество **букв** (только буквы, без пробелов и знаков)
2. Количество **цифр**
3. Количество **пробелов**

Пример: Ввод: Hello 123 World!

Вывод: Букв: 10, Цифр: 3, Пробелов: 2

Задача --- Анализ строки (решение)

Решение

```
1 s = input()
2 letters = 0
3 digits = 0
4 spaces = 0
5 for ch in s:
6     if ch.isalpha():      # буква?
7         letters += 1
8     elif ch.isdigit():    # цифра?
9         digits += 1
10    elif ch == " ":       # пробел?
11        spaces += 1
12 print(f"Букв: {letters}, Цифр: {digits}, "
13       f"Пробелов: {spaces}")
```

Задача --- Анализ строки (разбор)

Трассировка (вход: "Hi 5!"):

ch	тип	let	dig	sp
H	буква	1	0	0
i	буква	2	0	0
" "	пробел	2	0	1
5	цифра	2	1	1
!	другое	2	1	1

Вывод: Букв: 2, Цифр: 1,
Пробелов: 1

■ Ключевая идея

Три счётчика в одном цикле ---
каждый символ проверяется
цепочкой if/elif.

Это комбинация паттернов:

- «Счётчик» --- считаем события
- «Фильтр» --- проверяем условие
- «Классификация» --- разделяем на группы

■ Применимость

Один цикл может считать **сколько угодно** разных счётчиков!

Вложенные циклы for

Теория

Один цикл for можно поставить **внутри** другого. Внутренний цикл выполняется **полностью** на каждой итерации внешнего.

```
1 for i in range(1, 4):  
2     for j in range(1, 4):  
3         print(f"{i}x{j}={i*j}",  
4             end="  ")  
5     print()
```

1x1=1	1x2=2	1x3=3
2x1=2	2x2=4	2x3=6
3x1=3	3x2=6	3x3=9

Как это работает?

1. $i = 1$: j пробегает 1,2,3
2. $i = 2$: j пробегает 1,2,3
3. $i = 3$: j пробегает 1,2,3

Всего итераций: $3 \times 3 = 9$

Общее правило:

$N \times M =$ всего итераций

Задача 5 --- Прямоугольник из звёздочек (условие)

Задача

Выведите прямоугольник из символов * размером $n \times m$ (n --- строки, m --- столбцы). Числа вводятся с клавиатуры.

Пример: Ввод: 3 5 $\rightarrow$ ***** / ***** / *****

Задача 5 --- Прямоугольник из звёздочек (решение)

Решение

```
1 n, m = map(int, input().split())
2 for i in range(n):          # строки
3     for j in range(m):      # столбцы
4         print("*", end="")  # безпереноса
5     print()                 # переноспослестроки
```

```
*****
*****
*****
```

Альтернатива (короче)

```
for i in range(n):
    print("*" * m)
```

Оператор `*` повторяет строку m раз!

break и continue

■ break --- остановить цикл

```
for i in range(1, 100):  
    if i * i > 50:  
        print("Найдено:", i)  
        break      # Выход из цикла
```

Вывод: Найдено: 8

break полностью прекращает выполнение цикла. Код **после** цикла выполнится.

■ continue --- пропустить итерацию

```
for i in range(1, 11):  
    if i % 3 == 0:  
        continue # пропускаем  
    print(i, end=" ")
```

Вывод: 1 2 4 5 7 8 10

continue пропускает **остаток текущей** итерации и переходит к следующей.

Бонус: enumerate() --- индекс + элемент

Теория

Иногда нужно знать и **порядковый номер**, и **значение** элемента одновременно. Для этого есть `enumerate()`.

```
1 fruits = ["яблоко", "банан",  
2           "вишня"]  
3 for i, f in enumerate(fruits):  
4     print(f"{i}: {f}")
```

0: яблоко

1: банан

2: вишня

С нумерацией от 1:

```
1 for i, f in enumerate(fruits,  
2                       start=1):  
3     print(f"{i}. {f}")
```

1. яблоко

2. банан

3. вишня

Шпаргалка: всё о цикле for

■ С range()

```
# N раз
for i in range(N):

# от a до b-1
for i in range(a, b):

# шагом s
for i in range(a, b, s):

# обратно
for i in range(N, 0, -1):
```

■ Со списком

```
# перебор
for x in [1, 2, 3]:

# индексом
for i, x in enumerate(lst):

# сумма
s = 0
for x in lst:
    s += x
```

■ Дополнительно

```
# строкой
for ch in "abc":

# break
for ...:
    if ...: break

# continue
for ...:
    if ...: continue

# вложенный
for i in range(n):
    for j in range(m):
```

Цикл for в Python

Неумоин Павел Дмитриевич

6 апреля 2026 г.